Neural Networks: From Computation to Learning

A concise, text-based educational sample for testing source-grounded concept extraction, citations, active recall, and prerequisite reasoning in Synapse.

Learning objective	Understand how layered functions produce predictions and how gradients update their parameters.
Core prerequisites	Loss functions, computational graphs, derivatives, and the chain rule.
Key diagnostic idea	Weak understanding of the chain rule can create downstream risk in backpropagation and vanishing gradients.

1. Foundations of a Neural Network

1.1 Layered computation

A feed-forward neural network is a composition of functions. Each layer transforms an input with learned weights and biases, then usually applies a nonlinear activation. If every layer were linear, the whole stack would collapse into one linear transformation. Nonlinear activation functions let the network represent curved decision boundaries and more complex mappings.

1.2 Loss functions

Training needs an objective. A loss function measures the discrepancy between a prediction and its target. The average loss over training examples is a scalar quantity, so its derivative with respect to each parameter tells us how a small parameter change would affect prediction error.

Why it matters: the loss provides the terminal objective from which backpropagation begins.

2. Computational Graphs and Backpropagation

2.1 Computational graphs

A computational graph represents operations as nodes and dependencies as directed edges. During the forward pass, intermediate values are computed and retained. During reverse differentiation, the graph is traversed backward so local derivatives can be reused instead of recomputing every complete derivative independently.

2.2 The chain rule

The chain rule differentiates a composition by multiplying local derivatives along its dependency path. For a composition y = f(g(x)), the derivative is dy/dx = df/dg times dg/dx. A multilayer network is a deep composition, so the chain rule is the mechanism that connects a change in an early weight to the final loss.

2.3 Backpropagation

Backpropagation is an efficient reverse-mode differentiation procedure. It starts at the loss and propagates gradients backward through hidden layers. At each operation, an incoming gradient is multiplied by the operation's local derivative. Reusing intermediate derivatives makes it possible to compute gradients for many parameters in one backward traversal.

Backpropagation computes gradients; gradient descent uses those gradients to update parameters. The two ideas are related but not identical.

3. Optimization and Gradient Pathologies

3.1 Gradient descent and learning rate

The gradient points toward the steepest local increase of the loss. Gradient descent therefore subtracts a learning-rate-scaled gradient from the current parameters. A learning rate that is too large can overshoot useful regions or diverge, while a rate that is too small can make learning unnecessarily slow.

3.2 Mini-batch training

A mini-batch estimates the full-data gradient from a subset of examples. This reduces memory and computation per update while introducing noise. The noise can vary the path through parameter space, but each update still relies on gradients produced by backpropagation.

3.3 Vanishing gradients

In a deep network, backpropagation repeatedly multiplies derivatives through many layers. When many of those derivatives are smaller than one, their product can become extremely small. The gradient arriving at early layers then carries little learning signal, so those layers update slowly. This is the vanishing-gradient mechanism.

Prerequisite connection: explaining vanishing gradients requires both the chain rule and the backward flow of gradients through the computational graph.

Review questions

1. Why does a multilayer neural network need nonlinear activation functions?
2. How does the chain rule enable backpropagation through hidden layers?
3. Why are backpropagation and gradient descent not the same operation?
4. What repeated mathematical operation causes gradients to vanish?